Evaluating Graph-Conditioned Generative AI for IoT Sensor Fault Detection and Repair

Anonymous Author(s)

Abstract. A temperature sensor reporting a sudden rise may indicate overheating or a faulty reading. Operators need to identify which measurements to rely on. We present a graph-conditioned generative AI method for locating inconsistent readings and proposing service. A neural normalizing flow uses sensor history and related measurements to generate plausible trajectories on GPUs. The method compares normal readings with explicit bias, drift, noise and stuck-sensor explanations, producing fault scores and weighted repair distributions. Controlled evaluation on a synthetic family, Intel Berkeley and SKAB separates detection, confidence and repair quality. The score improves mean average precision over PCA and even over the same flow's likelihood, with both paired primary intervals above zero. The smaller gain misses the declared practical margin, benefits over matched Gaussian models remain inconclusive, and supervised trees perform better. Calibration and repair failures restrict operational claims. Versioned graph evidence and an annotated sensor network support human views. This research provides an audited, reproducible framework for assessing generative fault diagnosis, identifying detection gains and exposing reliability limitations that anomaly rankings alone would not reveal.

Keywords: sensor data quality · conditional generation · fault localization · likelihood ratio · repair uncertainty

1 Introduction

Distributed sensors provide the measurements used by digital twins and shared data spaces. When several temperature sensors rise together, they may be reporting a genuine process change. Conversely, a sensor stuck at an ordinary value may be faulty. Operators therefore need more than an anomaly alert, they need to identify questionable readings and assess whether proposed corrections are needed.

We use generative AI to compare two explanations for each sensor interval, the recorded measurements are plausible, or a measurement fault has altered them. A graph conditioned flow generates possible reference trajectories from sensor history and related measurements. Evaluating these trajectories under specified corruption mechanisms produces a likelihood-ratio fault score and weighted repair distribution. Supporting sensors provide context, graph edges describe predictive associations without establishing causation.

H1 tests whether integrating these generated explanations improves sensor-interval average precision over development selected PCA and ordinary negative log likelihood from the same flow. H2 compares neural and Gaussian or probabilistic PCA models using

identical corruption assumptions. H3 evaluates useful repair coverage under a separately selected policy aiming at a controlled failure rate. Repair is a secondary endpoint.

The results support H1 in the three-family comparison while exposing limitations in confidence and repair reliability. The methodological value lies in separating these outcomes through matched baselines, numerical audits and tests of missing targets, unseen faults and corrupted context. The previous diffusion method is rerun on shared cases. Fresh synthetic trajectories provide new tests, injections into previously examined real recordings assess controlled fault behavior without establishing generalization to untouched situations.

2 Related Work

PCA residuals are established monitoring tools [8]. Probabilistic PCA and its mixture extension supply normalized distributions, generation and uncertainty [15,14]. Neural benefits must therefore be measured against these probabilistic repair alternatives. GANF combines recurrence, learned graphs and flows for anomaly detection [3]. MTGFlow studies dynamic graphs and entity-specific densities [19]. Recent work examines departures in conditional-flow latent dynamics [1]. We use unchanged author GANF classes with documented evaluation adapters. Our affine couplings follow established constructions [5]. Neither graph conditioning nor invertibility is new here.

Likelihood ratios for distributional anomalies have substantial precedent [12]. Bayesian sensor diagnosis predates neural flows [11], and particle filtering integrates uncertain latent system states when assessing sensor fault explanations [17]. A flow prior combined with a structured noise likelihood has also been used for inverse recovery [16] (these are close predecessors). Our study evaluates a particular combination of masked graph context, normalized short block corruption channels, marginal likelihood comparison and weighted repair.

CSDI and PriSTI generate conditional imputations [13,10]. ImDiffusion uses diffusion imputation for anomaly detection [2], and GDN uses graph based forecast deviations [4]. We retain GDN and documented diffusion comparators. The earlier procedure tested repairs against excluded witness measurements. The present conditional likelihood method does not inherit that independence claim. A source comparison matrix records implementation differences.

3 Conditional Fault Explanations

3.1 Sensor intervals and supporting context

Each sensor's recent readings are assessed against its history and related sensors. Every 32 steps, we examine a 64 step window and test each channel's final eight observations. Candidate h identifies the target block, with values $y_h \in \mathbb{R}^{D_h}$ and $D_h = 8$. The variable z_h represents a possible uncorrupted, but still noisy, version. All eight values must be available. Incomplete blocks are flagged and assigned a common lowest score, retaining their faults as misses (missing targets are not zeros).

The context C_h contains the information used to generate reference readings. To prevent copying the tested measurements, we hide the same eight-step interval in every channel from the target's physical source before computing features or graph messages. Earlier readings remain available. Training medians and interquartile scales normalize channels, missing context is masked, and duplicate measurements are represented once. With latent draws fixed, altering hidden values or copies must leave the context and generated samples unchanged, the target likelihood may change.

The frozen graph G identifies supporting channels through predictive associations selected using training and development data. Edges record direction, lag, units, training correlation and development support. Up to three incoming channels from distinct other sources provide context. A shared gated recurrent unit summarizes histories, channel embeddings identify sensors, and available graph messages are averaged. These inputs form $c_h = \mathrm{Encoder}_\psi(C_h, G)$. Only information available at the decision time is used. Lags and recurrent steps count observations, whose elapsed durations can differ in irregular recordings.

3.2 Neural density and actual generation

An invertible conditional flow maps a target block to a standard normal latent vector. With f_θ denoting the forward map,

$$\log p_\theta(y_h \mid c_h) = \log p_0(f_\theta(y_h; c_h)) + \log \left| \det \frac{\partial f_\theta}{\partial y_h} \right|. \tag{E1}$$

The conditioner is fixed when this target Jacobian is evaluated. Alternating affine couplings have triangular Jacobians and bounded neural log scales. A conditional location and scale transform precedes them. Generation applies the inverse on the GPU,

$$\epsilon_h^{(m)} \sim \mathcal{N}(0, I), \qquad z_h^{(m)} = f_\theta^{-1}(\epsilon_h^{(m)}; c_h). \tag{E2}$$

These are numerical sensor trajectories. They enter both the fault score and the repair calculation below. The reference training objective is

$$\mathcal{L}_{\mathrm{density}} = -\frac{1}{N} \sum_{h=1}^{N} \log p_\theta(y_h \mid c_h). \tag{E3}$$

Reference denotes the declared training population, not certified correct instrumentation. Saved records contain masks, contexts, latent draws, generated intervals and model hashes for representative production cases.

3.3 Normalized corruption channels

The normal explanation uses the flow density directly,

$$p_{\mathrm{normal},h}(y_h \mid C_h, G) = p_\theta(y_h \mid c_h). \tag{E4}$$

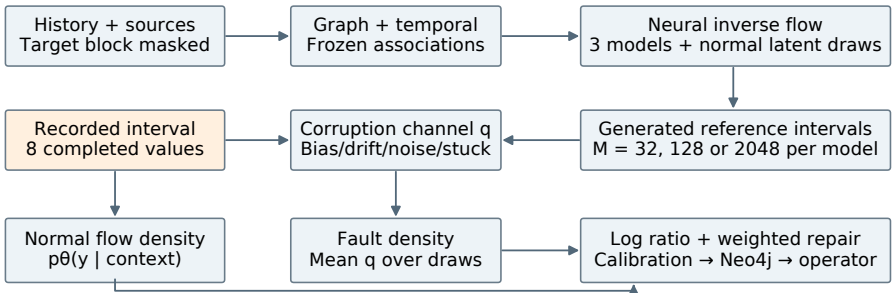


Fig. 1. Executed dataflow. The recorded target reaches density and corruption evaluation but is removed from the generation context. Generated trajectories directly enter the marginal fault density and the weighted repair. Graph persistence stores evidence references after numerical scoring.

The fault explanation uses a normalized mixture

$$q_\phi(y \mid z) = \sum_{k=1}^{4} \pi_k \mathcal{N}(y; H_k z, R_k), \qquad \pi_k = \tfrac{1}{4}. \tag{E5}$$

For bias, drift and excess noise, $H_k = I$. For a stuck reading, $H_k = \mathbf{1}e_1^T$, so the uncertain stuck level is centered on the first generated reference value. Let $s \in \{1, 2\}$ be the development-selected scale and $r = 0.25$ the fixed noise floor. The covariances are

$$R_{\text{bias}} = s^2\mathbf{1}\mathbf{1}^T + r^2 I, \qquad R_{\text{drift}} = s^2 t t^T + r^2 I,$$
$$R_{\text{noise}} = s^2 I, \qquad R_{\text{stuck}} = s^2\mathbf{1}\mathbf{1}^T + r^2 I.$$

Here t is the actual target timestamp ramp rescaled to $[-1, 1]$. Bias and drift amplitudes are shared across the block. Independent amplitudes at every coordinate would be different models. Positive floors make each covariance positive definite. Cholesky based determinants and quadratic forms produce correct Gaussian densities. Dropout is an availability event and is not represented by a numeric corruption density.

The marginal fault density and log ratio are

$$p_{\text{fault},h}(y_h \mid c_h) = \int q_\phi(y_h \mid z) p_\theta(z \mid c_h)\, dz, \tag{E6}$$

$$S_h = \log p_{\text{fault},h}(y_h \mid c_h) - \log p_\theta(y_h \mid c_h). \tag{E7}$$

Positive S_h favors the declared fault model. It is not itself a calibrated probability or proof of sensor failure. With generated trajectories from (E2),

$$\widehat{S}_h = \log\left[\frac{1}{M}\sum_{m=1}^{M} q_\phi(y_h \mid z_h^{(m)})\right] - \log p_\theta(y_h \mid c_h). \tag{E8}$$

We use stable log-sum-exp arithmetic. The logarithm of the average density is essential. Averaging log densities or taking the best generation does not implement this integral.

The density estimate is unbiased under independent prior sampling, but its logarithm generally has downward bias.

For E equally weighted trained models with M samples each,

$$\begin{aligned}\widehat{S}_{h,E} = \mathrm{LME}_{e,m}\big[\log q_\phi(y_h \mid z_h^{(e,m)})\big] \\ - \mathrm{LME}_e\big[\log p_{\theta_e}(y_h \mid c_h)\big],\end{aligned} \tag{E9}$$

where $\mathrm{LME}(a_1, \ldots, a_n) = \log(\sum_i \exp(a_i)/n)$. The ordinary flow comparator is the negative second term's underlying mixture log density, using exactly the same checkpoints. An average of member log ratios is a separately labeled ablation.

3.4 Repair distribution and numerical adequacy

Bayes' rule under the fault explanation gives

$$p(z \mid y_h, c_h, H_{1,h}) = \frac{q_\phi(y_h \mid z)\, p_\theta(z \mid c_h)}{p_{\mathrm{fault},h}(y_h \mid c_h)}. \tag{E10}$$

For a uniform ensemble, we normalize $q_\phi(y_h \mid z_h^{(e,m)})$ over all generated members and samples to obtain weights $w_{e,m}$. Weighted means evaluate squared-error repair, and weighted medians and marginal quantiles are also kept. The reference truth never chooses a sample. A mean need not preserve a physical mode.

$$\mathrm{ESS}_h = \left(\sum_{e,m} w_{e,m}^2\right)^{-1}. \tag{E11}$$

An ESS below 16 prevents a repair recommendation but leaves the detection score in the evaluated population. Low ESS identifies poor importance sampling resolution, not necessarily large physical uncertainty. Repeated sampling seeds and a finite draw grid diagnose this limitation separately from differences between trained models.

We apply a proposed action to the full evaluated window. With normalized squared loss L summed over observed cells and divided by eight, reference $y_j^\star$, acceptance a_j and useful-improvement threshold $\delta = 0.01$,

$$\begin{aligned}\Delta_j &= L(y_j, y_j^\star) - L(\mathrm{repair}_j, y_j^\star), \\ F_j &= \mathbf{1}\{\text{wrong target or } \Delta_j \le \delta\}, \\ \text{Coverage} = \frac{\sum_j a_j}{n}, &\quad \text{Risk} = \frac{\sum_j a_j F_j}{\sum_j a_j}.\end{aligned} \tag{E12}$$

A 'wrong target action' includes damage to previously correct measurements. Harmful edits have $\Delta_j < 0$. No accepted action gives zero coverage and undefined risk. On real datasets the repair reference is the pre-injection recording, not unknown physical truth.

4 Mathematical Checks and Limits

4.1 Probability checks and false alarms

To compare normal and faulty readings, both explanations must assign total probability one to the same target space. In (E6), the nonnegative integrand allows us to integrate over y first. Each corruption channel integrates to one, leaving $\int p_\theta(z \mid c)\, dz = 1$. The resulting fault model is therefore a valid density. Both explanations use the same continuous approximation for quantized readings. Suppose the normal model exactly describes normal measurements and the fault model assigns no probability to regions excluded by the normal model. Then

$$\mathbb{E}_{H_0}[\exp(S_h) \mid c_h] = \int p_{\text{fault}}(y \mid c_h)\, dy = 1.$$

Markov's inequality gives $\Pr_{H_0}(S_h \geq \log(1/\alpha) \mid c_h) \leq \alpha$: under these assumptions, a fixed candidate exceeds this threshold with probability at most α. Learned models can violate the assumptions through density error, corrupted context or distribution shift. Searching across many candidates also increases opportunities for false alarms, so the largest score does not inherit this bound. Averaging valid ratios with fixed nonnegative weights summing to one preserves the expectation, observation dependent weights require separate justification. We therefore calibrate the actual maximum score used by the alarm procedure.

4.2 Gaussian reference and measurement units

A Gaussian example provides an exact answer against which to check the implementation. Let reference readings follow $z \sim \mathcal{N}(\mu, \Sigma)$ and let independent corruption add $\eta \sim \mathcal{N}(0, R)$. The recorded values $y = z + \eta$ then follow $\mathcal{N}(\mu, \Sigma + R)$. Comparing this fault density with the normal density gives

$$S = -\tfrac{1}{2} \log \frac{|\Sigma + R|}{|\Sigma|} + \tfrac{1}{2}(y - \mu)^T [\Sigma^{-1} - (\Sigma + R)^{-1}](y - \mu).$$

The corresponding repair distribution has mean $\mu + \Sigma(\Sigma + R)^{-1}(y - \mu)$ and covariance $\Sigma - \Sigma(\Sigma + R)^{-1}\Sigma$. For the affine channels in (E5), the fault density instead uses mean $H_k\mu$ and covariance $H_k \Sigma H_k^T + R_k$. Conditional PPCA evaluates these Gaussian integrals and posterior mixture weights exactly, providing a baseline with the same fault assumptions but without neural sampling.

Changing units should not change the score when both models are transformed consistently. Under an invertible change $u = g(y)$, both densities acquire the factor $|\det Dg^{-1}(u)|$, which cancels in their ratio. This property concerns the same distributions described differently, retraining models or changing corruption assumptions can change the results. Numerical checks include non-diagonal affine transformations.

4.3 Limits of fault identification

Two sensors can agree and still leave the cause uncertain. If both measure x_t, a genuine process change of a_t produces $(x_t + a_t, x_t + a_t)$. The same readings arise if the process stays unchanged but both sensors acquire an additive fault a_t. With identical available observations and context, no detector can distinguish these explanations without additional information. Our stress study stores both scenarios with identical observations and different latent truth. Their ambiguity is known from construction, labeling these cases does not establish that the detector recognizes ambiguity immediately.

5 Experimental Protocol

5.1 Datasets, independence and candidates

Table 1 summarizes the evaluated sensor intervals. S32L, S32N and S64N are 32 channel linear, 32 channel nonlinear and 64 channel nonlinear configurations of one coupled simulator, not independent datasets. New test trajectories separately record the physical process, uncorrupted noisy measurements and injected faults. The last two of twelve final trajectories use the predefined operating regime change.

Intel provides 48 channels from twelve sensor nodes measuring temperature, humidity, light and voltage [7]. Completed five minute bins retain missing values without interpolation, chronological splits and gaps are preserved. SKAB has eight sensor/process channels split by experiment [9]. Native process events are evaluated separately, controlled faults use only reference windows without these events. Scaling uses training data, and physical source groups remain static.

All real partitions informed the earlier study, new injections therefore test familiar recordings, not untouched environments. Test observations remain excluded from training. Earlier synthetic training, development and calibration sources are reused, preserving the previous comparator's training opportunity.

Each reference window, sampled by source block, produces one unchanged copy and four fault copies. Development and calibration use bias, drift, excess noise and stuck readings. Each final reference receives two known family faults and two 'held out family faults', the latter rotate across spike, scale, replay and delay. Severities are newly sampled from the fixed range. Fault labels require a change in an observed target value. Wholly unobservable injections are recorded as availability outcomes. Primary injections leave other physical sources unchanged, but their original recordings may still contain faults.

5.2 Development selection and comparators

Matched comparisons test what drives detection gains, fault scoring, neural modeling or labeled fault examples. The flow grid combines width/layer pairs $(32, 4)$ and $(64, 6)$ with context lengths 32 and 56. Each fit uses 600 AdamW steps, batches of 128, learning rate 0.001 and weight decay 0.0001. Normal development likelihood selects checkpoints and capacity. Three fixed seeds form one density mixture ensemble, not three ensemble repetitions. S32N and SKAB learning curves use one quarter, one half and all training blocks, (Table 7 reports model and training sizes).

Table 1. Evaluated populations. Blocks gives training/final source counts. Candidates are sensor intervals. Eligibility requires all eight target observations, intervals with missing readings remain in the primary evaluation.

Config.	Channels	Blocks	Cases	Candidates	Faults	Eligible
S32L	32	12/12	320	10,240	256	100.0%
S32N	32	12/12	320	10,240	256	100.0%
S64N	64	12/12	320	20,480	256	100.0%
Intel	48	14/6	220	10,560	138	65.0%
SKAB	8	7/10	320	2,560	255	100.0%

Development selects corruption scale 1 or 2 and sampling budget $M \in \{32, 128, 512, 2048\}$. We choose the smallest budget with a 95th-percentile absolute score difference at most 0.25 and rank correlation at least 0.98 against 2048 draws, defaulting to 2048 if needed. These checks assess numerical stability without guaranteeing exact integration. Separate sampling seeds test repeatability. All choices, data hashes and numerical source hashes are frozen before final scoring.

The identical flow's negative log likelihood isolates the scoring contribution. PCA compares current-only and four-lag reconstructions, with valid ranks from 2, 4, 8 and 16 selected by development candidate AP. Scores average target-channel squared residuals across eight steps, window maxima and full residual norms are stored separately. Conditional PPCA and two component mixture PPCA test neural modeling against Gaussian alternatives with identical graph context and q_ϕ. Normal development likelihood selects ranks 2, 4 or 8. All channel PPCA is reported separately. Missing context is marginalized during evaluation but mean filled during fitting, a limitation when missingness is informative.

GANF uses the authors' unchanged recurrent, graph and autoregressive flow classes at a pinned revision, widths 16 and 32, ten graph optimization iterations and forty steps per iteration. The adapter excludes missing target losses and same source adjacency, aggregating the final eight coordinate densities. GANF can use preceding target readings within that interval and fills missing context without a mask. We report remaining acyclicity residuals rather than assuming exact convergence. MTGFlow was inspected but not run within budget.

A gradient boosted classifier tests the value of explicit fault labels, using shape, earlier history, PCA residual and availability features. It trains on the four development fault types injected into training windows. The earlier diffusion witness, GDN, backbone residual and documented DiffAD adaptation [18] are rerun on shared manifests. The witness retains its top-four screen, ignored candidates receive a score floor, keeping excluded faults as misses. Earlier window level results remain archived separately.

5.3 Confidence and decision policy

Three disjoint sets of whole calibration blocks separate probability estimation, alarm calibration and repair selection. Labeled blocks fit each comparator's logistic probability calibration using a robustly scaled score and scorable indicator. Normal blocks calibrate

each window's maximum candidate score. For n normal calibration windows,

$$p_{\text{window}} = \frac{1 + \sum_{b=1}^{n} \mathbf{1}\{S_{\max,b} \geq S_{\max,\text{new}}\}}{n+1}.$$

The minimum $1/(n+1)$ limits attainable alarm levels; dependence and distribution shift further restrict guarantees.

Remaining blocks select a repair probability threshold, maximizing coverage over a fixed grid subject to ten or more actions and a one-sided 95% Wilson failure upper bound ≤ 0.10. If none qualifies, the policy abstains. Under dependence, this is a calibration diagnostic, not a deployment guarantee. Test failures and harmful repairs are reported. Marginal CRPS, interval coverage and width, and joint energy score assess repair distributions [6]. Flow energy estimates use 512 weighted sample pairs.

5.4 Endpoints and uncertainty

Primary AP measures fault ranking across all sensor intervals, including clean windows and unaffected candidates. For comparator b, the registered contrast is

$$\Delta_b = \tfrac{1}{3} \sum_{d \in \{\text{synthetic,Intel,SKAB}\}} (\text{AP}_{\text{ratio,d}} - \text{AP}_{b,d}),$$

The three synthetic configurations are averaged first, giving each dataset family equal weight. The practical improvement margin is 0.02 absolute AP. Directional H1 requires positive evidence against both PCA and same flow NLL.

Uncertainty uses 1000 paired whole block bootstrap replicates, resampling synthetic trajectories, Intel time blocks and SKAB experiments while retaining correlated candidates and injected copies. Paired 32-channel synthetic seeds share draws. Individual 97.5% intervals for the two primary contrasts support a Bonferroni simultaneous 95% statement. Dataset intervals are descriptive, few real blocks limit stability. Training seeds are not independent test subjects. Secondary metrics assess localization, window detection, calibration, review precision, repairs and measured costs.

6 Results

6.1 Primary comparison and neural contribution

Across 1,500 cases and 54,080 candidate intervals, the generative fault score achieves family macro AP of 0.438, compared with 0.307 for PCA and 0.423 for same-flow NLL. Paired gains are +0.131 [+0.093, +0.147] and +0.015 [+0.005, +0.027], respectively. Both simultaneous primary intervals are positive, supporting directional H1 under the locked protocol. The comparison with the identical flow demonstrates the added detection value of explicit fault explanations. Its smaller gain remains below the chosen 0.02 practical margin. Improvements over PCA extend to S64N and both real data families, with SKAB contributing most of the macro gain. This identifies settings where the generative testing framework improves fault ranking. PCA remains stronger in both

Table 2. Sensor-interval average precision over the full candidate population. The synthetic configurations form one family in the macro-average. All methods use identical cases and complete-target eligibility.

Method	S32L	S32N	S64N	Intel	SKAB	Macro
Flow + corruption	0.589	0.633	0.608	0.075	0.628	0.438
PCA residual	0.680	0.659	0.541	0.018	0.275	0.307
Same-flow NLL	0.588	0.632	0.600	0.058	0.604	0.423
Graph PPCA + corruption	0.654	0.629	0.594	0.032	0.494	0.384
Mixture PPCA + corruption	0.654	0.626	0.584	0.028	0.473	0.374
All-channel PPCA + corruption	0.722	0.697	0.474	0.019	0.394	0.348
Official GANF	0.532	0.473	0.290	0.015	0.230	0.225
Supervised trees	0.708	0.700	0.641	0.366	0.686	0.578
Legacy diffusion witness	0.494	0.632	0.449	0.014	0.188	0.242
GDN residual	0.305	0.416	0.429	0.024	0.265	0.224
Diffusion residual	0.665	0.623	0.439	0.016	0.379	0.324
DiffAD adaptation	0.482	0.443	0.299	0.029	0.481	0.306

Table 3. Known and held-out fault families, each pooled with clean windows. Entries give flow-ratio AP / PCA AP. Held-out mechanisms were absent from channel modeling and classifier development.

Config.	Known	Held out
S32L	0.785/0.918	0.371/0.471
S32N	0.888/0.928	0.365/0.411
S64N	0.875/0.795	0.325/0.282
Intel	0.095/0.018	0.035/0.013
SKAB	0.790/0.318	0.420/0.208

32-channel synthetic settings, with negative paired intervals for the ratio. Intel's absolute performance remains limited: only 65.0% of target blocks are eligible, and 37 changed targets remain unscorable misses.

The ratio improves the point estimate over matched PPCA by +0.054 [-0.007, +0.082]. H2 remains inconclusive because this and the mixture-PPCA interval span zero. Gaussian controls are particularly effective in the linear setting. Supervised trees lead every configuration, with a macro advantage of 0.140 and ratio-minus-tree interval [-0.169, -0.114], providing a strong benchmark when labeled faults are available. GANF and rerun legacy methods broaden the comparison (Table 2). Top-one ratio attribution is 0.660 on S32N, 0.065 on Intel and 0.631 on SKAB. Although held-out mechanisms substantially reduce AP, the ratio retains its advantage over PCA in S64N and both real-data families (Table 3).

6.2 Calibration, review and repair

The ratio improves Brier score over PCA in all synthetic configurations and SKAB, supporting improved probability prediction on these datasets. Intel exposes a calibration

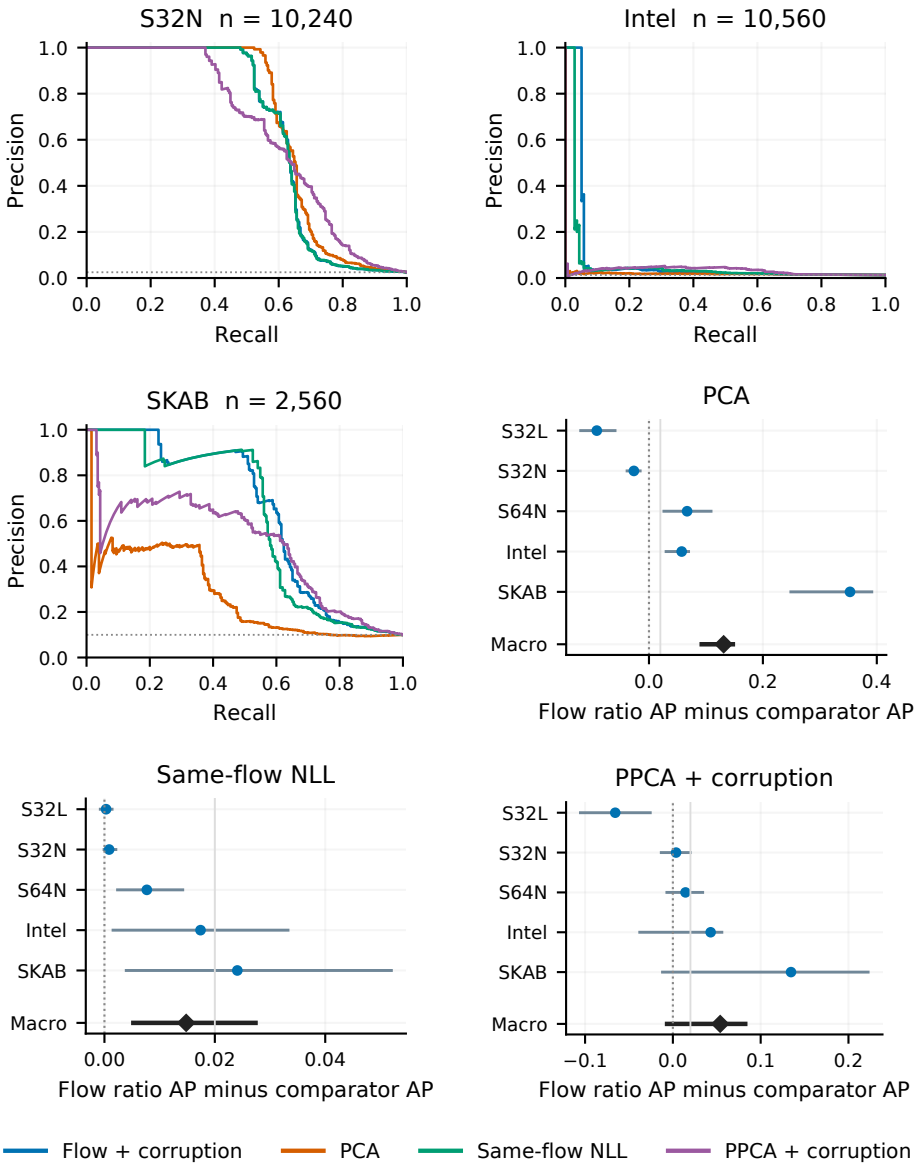


Fig. 2. Sensor-interval precision–recall curves and paired AP differences. All configurations contribute to the three-family macro contrast. Dataset intervals are 95%. The two primary macro intervals are 97.5% individually.

Table 4. Flow ratio confidence and selected repair policy on final cases. Alarms use the separately calibrated window maximum at level 0.10. Repair counts cover all evaluation cases. A dash denotes undefined risk because no action was accepted.

Config.	Brier	Log loss	Min. p	False alarms
S32L	0.012	0.066	0.027	6/64
S32N	0.012	0.065	0.027	10/64
S64N	0.007	0.037	0.027	4/64
Intel	0.068	0.441	0.059	79/82
SKAB	0.053	0.276	0.026	3/65

Config.	Recall	Repairs	Failed	Harmful
S32L	0.590	122	0/122	0
S32N	0.691	134	0/134	0
S64N	0.648	125	0/125	0
Intel	0.993	0	–	0
SKAB	0.463	123	24/123	24

limit: Brier score is 0.068 with block interval [0.054, 0.082], versus PCA's 0.017. Its minimum window p-value of 0.059 cannot resolve 1% or 5% alarm alpha levels. At level 0.10, 79 of 82 unchanged cases alarm, so improved AP does not establish usable Intel alarms. Normal cases include injections that changed no observed coordinate.

Synthetic repairs provide a positive result, the flow policy accepts useful corrections without observed failures. S64N accepts 125 of 320 cases, compared with no PCA recommendations and 129 for PPCA. PPCA overall achieves greater coverage. Zero observed failures does not establish zero population risk, particularly with dependent blocks. Intel abstains throughout, leaving recommendation risk undefined.

On SKAB, 123 accepted repairs have 24 harmful failures, risk 0.195 with block interval [0.000, 0.400], versus 0.371 for PCA and 0.183 for PPCA. Observed risk is lower than PCA's, but exceeds the calibration target; H3 remains unestablished. Matched PPCA also has lower CRPS and energy score throughout (Table 5), while wider flow intervals do not ensure reliable real-data coverage. Successful and failed examples (Figure 4) make these trade-offs inspectable. The framework therefore distinguishes detection gains from repair quality and decision reliability. Calibrated probabilities reflect the designed injection prevalence, not operational physical failure probabilities.

6.3 Ablations and adverse conditions

Graph context contributes a clear detection gain, macro AP improves by 0.053 [+0.028, +0.077] over own history alone. Integration adds 0.003 [-0.007, +0.020] over replacing generated references with their prior mean, leaving its additional benefit inconclusive. Family macro AP is 0.427 for one member and 0.468 when member log ratios are averaged instead of mixing densities. All 'leave one corruption channel out' results are retained without revising the fault prior after testing, scale sensitivity uses development results.

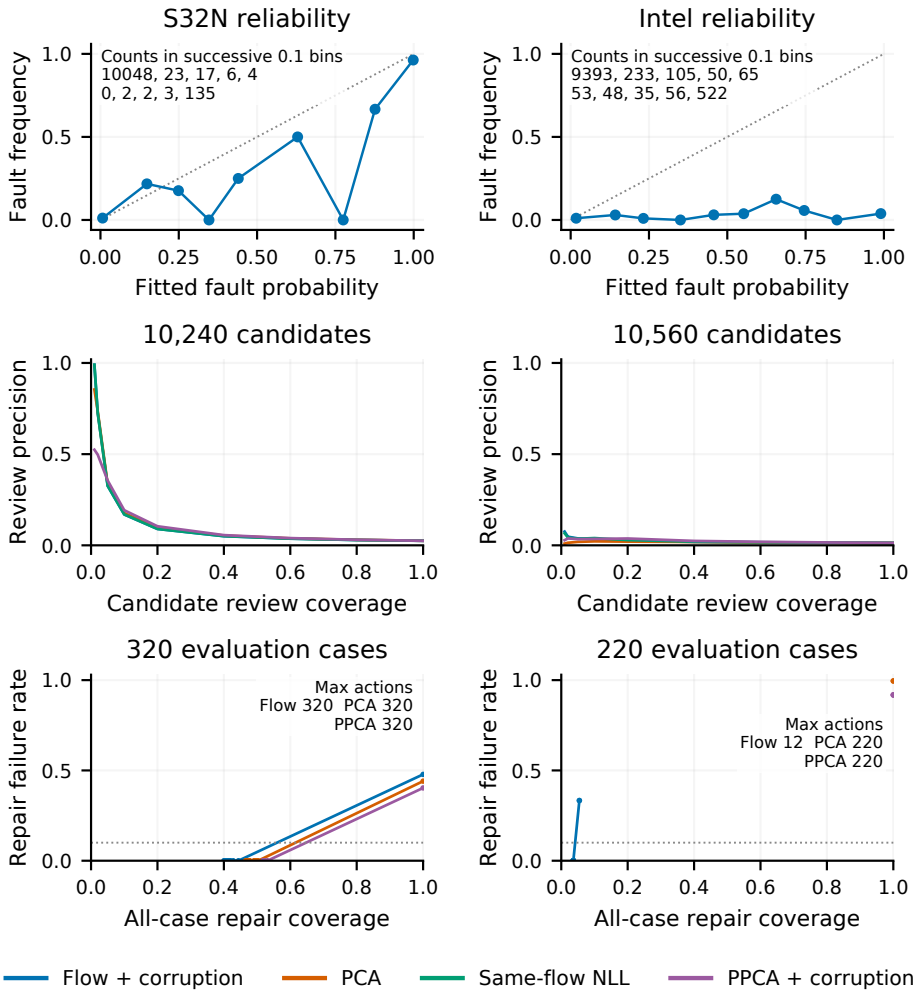


Fig. 3. Reliability, review precision and repair risk. Reliability insets count successive bins, five per line. Risk insets count actions at maximum coverage. Curves are threshold sweeps. Table 4 gives calibration-selected actions. Zero coverage has undefined risk.

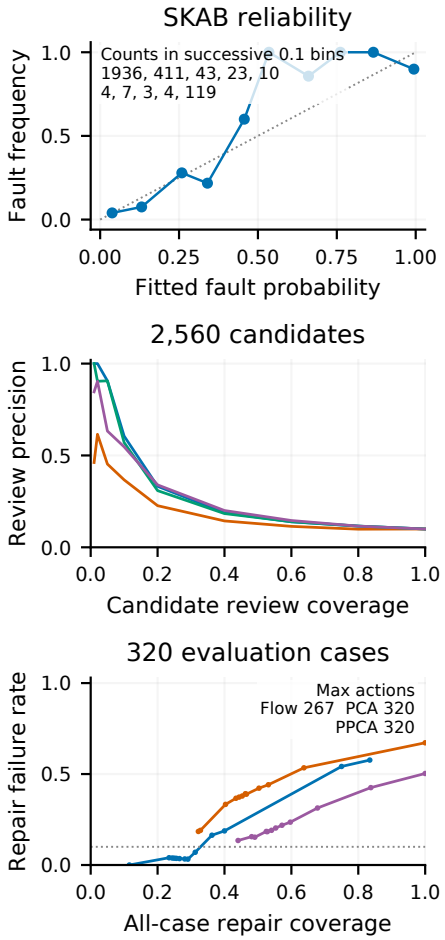


Fig. 3. Continued.

Table 5. Oracle-target repair distributions on eligible injected faults. Width and scores use training-normalized measurement units. Interval coverage is measured for nominal 90% marginal intervals. Lower CRPS and energy score are better.

Config.	Prior	CRPS	Coverage	Width	Energy
S32L	Flow	0.157	0.930	1.328	0.468
S32L	PPCA	0.024	0.961	0.169	0.081
S32N	Flow	0.039	0.975	0.341	0.129
S32N	PPCA	0.027	0.909	0.167	0.092
S64N	Flow	0.084	0.912	0.488	0.285
S64N	PPCA	0.069	0.875	0.329	0.238
Intel	Flow	3.494	0.399	1.154	9.905
Intel	PPCA	0.827	0.344	0.232	2.454
SKAB	Flow	0.391	0.775	1.202	1.249
SKAB	PPCA	0.267	0.778	0.602	0.877

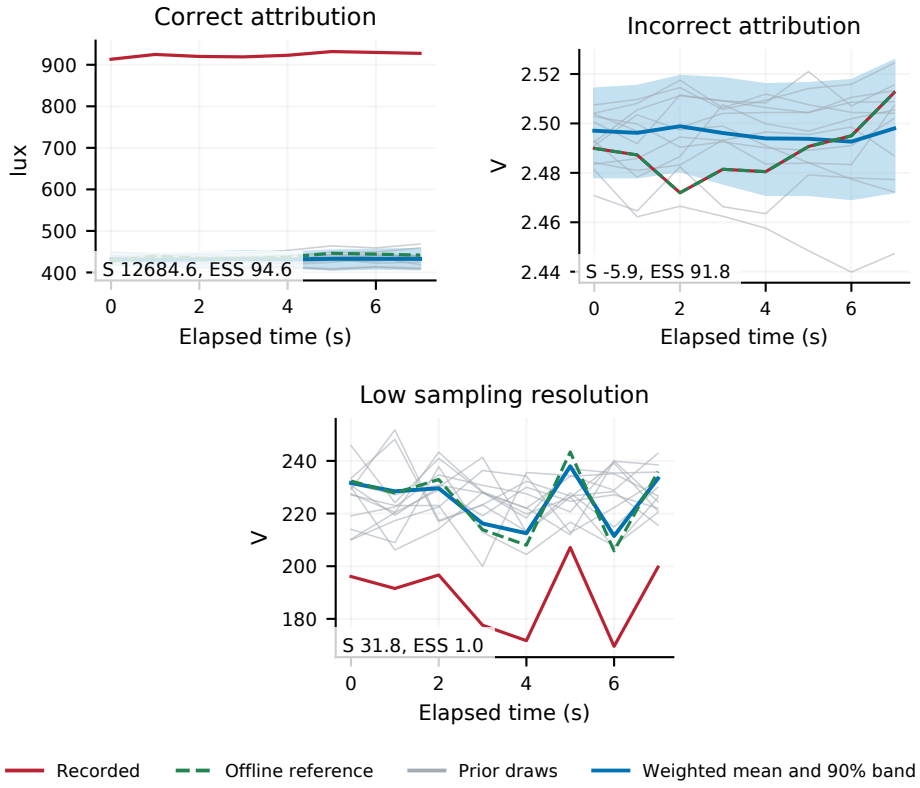


Fig. 4. Cases chosen by frozen success, failure and numerical-adequacy rules. Gray trajectories are GPU generations. Blue bands are weighted 90% marginal intervals. Reference truth is only for offline evaluation. Low ESS limits repair reliability. Case IDs and hashes are distributed with the figure.

Table 6. Registered stress diagnostics on twelve source windows per configuration. First three columns give primary-target top-one rates. The last gives measurement alarms within a graph-only error affecting half the stored associations. This is cross-type confusion, not association diagnosis.

Config.	Single	Half missing	Half faulty	Graph alarm
S32L	1.000	1.000	0.000	0.083
S32N	1.000	0.917	0.000	1.000
S64N	1.000	0.750	0.083	1.000
Intel	0.083	0.250	0.167	0.750
SKAB	0.833	0.750	0.167	0.250

The audits expose numerical limits alongside these gains. Even at 2048 draws per member, SKAB's 95th percentile repeated seed score change is 0.952, so this finite reference does not establish exact integration. Low-ESS cases retain detection scores but abstain from repair. Missing support affects attribution (Table 6). With multiple faults, selecting another faulty sensor can lower original target top one accuracy without being incorrect. When half the other S32N sources are corrupted, AP across all true targets is 0.946, under the changed fault prevalence. Canonicalizing known copies preserves scores exactly in every registered copy case.

Direct residuals provide a separate association diagnostic, achieving mean edge localization AP of 0.977 across twelve S32N cases with half the associations changed. Reading and association alarms can coincide or both remain silent. Cross type counts are saved; reading alarms do not prove false edges. The physical change counterexample clarifies an unavoidable ambiguity, identical readings can reflect a process change or shared sensor faults. All twelve S32N physical change cases trigger reading alarms at 0.10. On 224 native SKAB windows, including 78 process-event windows, AP is 0.536 for the ratio, 0.546 for NLL and 0.499 for PCA. These results assess process detection, not sensor fault diagnosis.

6.4 Numerical evidence and computation

The implementation passes eight scalar-reference check groups and forty local tests. Eighteen flow checks cover inverse/Jacobian agreement, Gaussian integrals and posteriors, normalized corruption, units, mixture arithmetic, masking, missingness, source copies, chunking and repair damage. A GPU pilot trained a 22,048-parameter flow on Gaussian simulations, changing validation NLL from 10.669 to 5.722 in 9.69 seconds.

The production audit reconstructs 107,620 calibration and final scores from saved component integrals, with maximum discrepancy 9.09×10^{-13}. Independent NumPy corruption densities and GPU model/draw replay check 30 full generation bundles. Maximum generation, normal density, corruption density and ratio discrepancies are 0, 0, 9.12×10^{-12} and 3.95×10^{-12}. Weighted posterior means agree within 1.97×10^{-12}; analytic and autograd Jacobian determinants agree within 3.55×10^{-15} for a trained model. Together, these checks link reported scores and repairs to the implemented equations, predictive quality is assessed through the benchmarks.

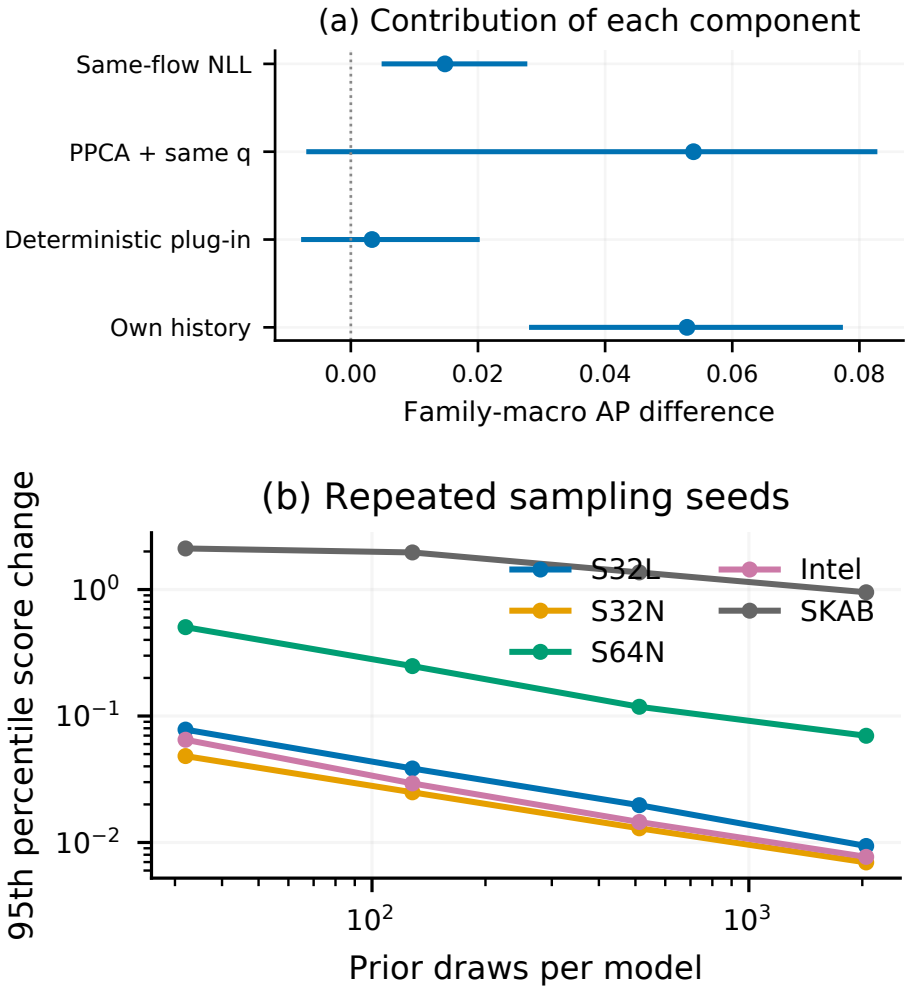


Fig. 5. Equation contributions and computational costs. The plugin replaces integration by the generated prior mean. Repeated seeds use fixed development cases. Panel (c) includes offline distribution metrics. Panel (d) measures operational scoring and repair without truth-dependent metrics.

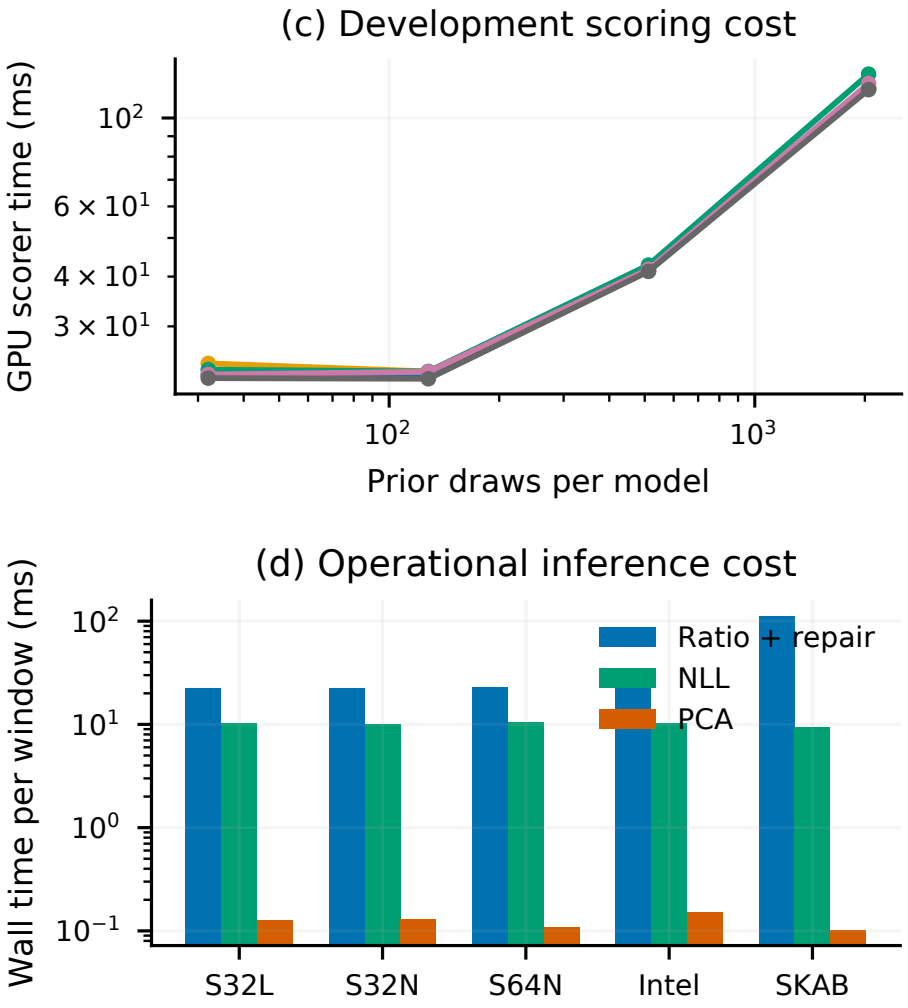


Fig. 5. Continued.

Table 7. Selected model size, training intervals and operational latency on the RTX PRO 4500. Times are medians of seven warm full window calls including repair summaries. Loading and offline truth metrics are excluded.

Config.	Params (k)	Train N	M	Ratio (ms)
S32L	96.9	11904	32	22.3
S32N	96.9	11904	32	22.7
S64N	97.4	23808	128	23.0
Intel	97.2	5072	32	22.5
SKAB	96.5	3272	2048	112.5

The study completed 70 neural fits, including the Gaussian pilot, in 10.8 recorded fitting minutes, excluding data preparation, Gaussian-baseline fitting, scoring and evaluation. Selected flows contain 96,512 to 97,408 parameters per member; peak allocation across registered neural fits is 0.49 GiB. Training the three selected members takes 34.5 to 36.9 seconds per configuration. Operational scoring, repair summaries and evidence transfer take 22.3 to 112.5 milliseconds per window (Table 7). Same flow NLL takes 9.3 to 10.5 milliseconds, versus 0.10 to 0.15 for PCA. PCA and PPCA run on the CPU. GANF acyclicity residuals of 1.53e-05 to 0.00776 record incomplete convergence within budget. Neural computation uses float32 and score arithmetic float64, without mixed precision. Independent replay exposed a float32 SciPy ensemble reduction, explicit promotion corrected the audit without changing predictions, models or the locked protocol. The failed audit and correction remain available.

7 Graph Evidence and Operator Review

Versioned Neo4j records make detection and repair evidence traceable across sensors, physical sources, windows, associations, models, fault hypotheses, generations, scores and repairs. Large arrays use content addressed files. Observations remain immutable, accept, reject and defer decisions are stored separately without automatically applying repairs.

A conventional 2-dimensional network presents named sensors and directed edges annotated with lag and training correlation. Red outlines identify disputed intervals, blue outlines supporting sources, and amber dashed edges separately assessed association disputes. Details show observed and generated trajectories, weighted repair intervals, ESS, benchmark probability and abstention. Reading scores do not establish per-edge certainty. The 23 saved cases cover first correct and incorrect attributions, inadequate sampling, constructed ambiguity and association review. Repeated Neo4j exports preserve entity counts. Automated browser checks verify evidence loading, node/edge selection, generated curves, hidden-by-default evaluation truth, artifact downloads, decision persistence and unchanged observations. These checks provide reproducible validation of the review interface; operator effectiveness remains to be evaluated.

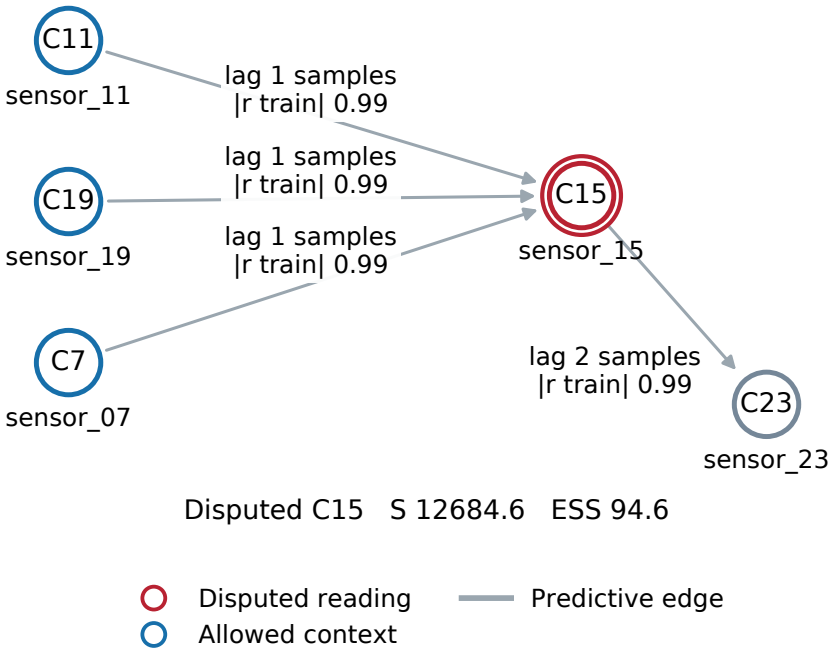


Fig. 6. Saved S32N sensor network case with a topology-selected neighborhood. Gray arrows denote predictive associations, blue outlines mark conditioning sources, and red marks the disputed interval. Its score does not certify individual edges.

8 Discussion

Generated alternatives connect fault detection to proposed corrections and inspectable graph evidence. Across three dataset families, macro AP increases by 0.131 over PCA and 0.015 over same-flow likelihood, with both primary intervals above zero. Supporting sensors improve detection over own history alone. SKAB drives most of the PCA gain, and the likelihood gain remains below the 0.02 practical margin.

Separate evaluation of ranking, confidence and repair shows where detection gains support useful actions. Synthetic repairs are beneficial without observed failures, but poor Intel calibration and harmful SKAB recommendations prevent dependable deployment claims. Matched Gaussian and mean-replacement controls leave neural and integration benefits unresolved; supervised trees perform better with labeled faults. These comparisons guide confidence calibration, repair selection and abstention.

The findings concern controlled faults in short, complete intervals. Missing measurements, simultaneous failures, irregular timing and process-versus-fault ambiguity limit generalization.

9 Conclusion

This work connects generative AI to a complete sensor-review workflow, generating plausible readings, testing fault explanations, proposing corrections and presenting traceable graph evidence. The evaluation establishes detection gains over PCA and same-model likelihood while identifying remaining confidence and repair challenges. Its contribution combines measured improvements with a reproducible framework for investigating when generative sensor diagnosis can support trustworthy decisions.

AI disclosure

OpenAI Codex[1] assisted with developing the software and grammar checking. It also assisted with the present manuscript's LaTeX conversion, anonymization and proofreading. The authors take responsibility for the research design, verification of results, references, manuscript and final content.

References

1. Baumgartner, D., de Souza da Silva, E., Urteaga, I.: Anomaly detection in time-series via inductive biases in the latent space of conditional normalizing flows. In: Proceedings of the 42nd Conference on Uncertainty in Artificial Intelligence. Proceedings of Machine Learning Research, vol. 337, pp. 448–468 (2026), https://proceedings.mlr.press/v337/baumgartner26a.html
2. Chen, Y., Zhang, C., Ma, M., Liu, Y., Ding, R., Li, B., He, S., Rajmohan, S., Lin, Q., Zhang, D.: ImDiffusion: Imputed diffusion models for multivariate time series anomaly detection. Proceedings of the VLDB Endowment **17**(3), 359–372 (2023). https://doi.org/10.14778/3632093.3632101

[1] OpenAI, Codex: https://openai.com/codex/ (accessed November 10, 2026).

3. Dai, E., Chen, J.: Graph-augmented normalizing flows for anomaly detection of multiple time series. In: International Conference on Learning Representations (2022)
4. Deng, A., Hooi, B.: Graph neural network-based anomaly detection in multivariate time series. Proceedings of the AAAI Conference on Artificial Intelligence **35**(5), 4027–4035 (2021)
5. Dinh, L., Sohl-Dickstein, J., Bengio, S.: Density estimation using Real NVP. In: International Conference on Learning Representations (2017)
6. Gneiting, T., Raftery, A.E.: Strictly proper scoring rules, prediction, and estimation. Journal of the American Statistical Association **102**(477), 359–378 (2007)
7. Intel Berkeley Research Lab: Intel lab data (2004), https://db.csail.mit.edu/labdata/labdata.html, accessed: September 30, 2026
8. Jackson, J.E., Mudholkar, G.S.: Control procedures for residuals associated with principal component analysis. Technometrics **21**(3), 341–349 (1979)
9. Katser, I.D., Kozitsin, V.O.: Skoltech anomaly benchmark (SKAB) (2020), https://github.com/waico/SKAB
10. Liu, M., Huang, H., Feng, H., Sun, L., Du, B., Fu, Y.: PriSTI: A conditional diffusion framework for spatiotemporal imputation. In: 2023 IEEE 39th International Conference on Data Engineering (ICDE). pp. 1927–1939 (2023). https://doi.org/10.1109/ICDE55515.2023.00150
11. Mehranbod, N., Soroush, M., Panjapornpon, C.: A method of sensor fault detection and identification. Journal of Process Control **15**(3), 321–339 (2005)
12. Ren, J., Liu, P.J., Fertig, E., Snoek, J., Poplin, R., DePristo, M.A., Dillon, J.V., Lakshminarayanan, B.: Likelihood ratios for out-of-distribution detection. In: Advances in Neural Information Processing Systems. vol. 32 (2019)
13. Tashiro, Y., Song, J., Song, Y., Ermon, S.: CSDI: Conditional score-based diffusion models for probabilistic time series imputation. In: Advances in Neural Information Processing Systems. vol. 34, pp. 24804–24816 (2021)
14. Tipping, M.E., Bishop, C.M.: Mixtures of probabilistic principal component analysers. Neural Computation **11**(2), 443–482 (1999)
15. Tipping, M.E., Bishop, C.M.: Probabilistic principal component analysis. Journal of the Royal Statistical Society, Series B **61**(3), 611–622 (1999)
16. Whang, J., Lei, Q., Dimakis, A.G.: Solving inverse problems with a flow-based noise model. In: Proceedings of the 38th International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 139, pp. 11146–11157 (2021), https://proceedings.mlr.press/v139/whang21a.html
17. Wright, M.A., Horowitz, R.: Particle-filter-enabled real-time sensor fault detection without a model of faults. In: 2017 IEEE 56th Annual Conference on Decision and Control (CDC). pp. 5757–5763 (2017)
18. Xiao, C., Gou, Z., Tai, W., Zhang, K., Zhou, F.: Imputation-based time-series anomaly detection with conditional weight-incremental diffusion models. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. pp. 2742–2751 (2023). https://doi.org/10.1145/3580305.3599391
19. Zhou, Q., Chen, J., Liu, H., He, S., Meng, W.: Detecting multivariate time series anomalies with zero known label. Proceedings of the AAAI Conference on Artificial Intelligence **37**(4), 4963–4971 (2023). https://doi.org/10.1609/aaai.v37i4.25623